

C# Review

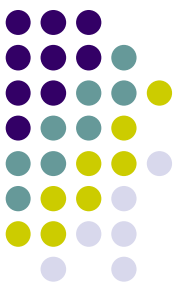
Architecture of Real-Time Systems
SE 456

Ed Keenan

7 January 2022

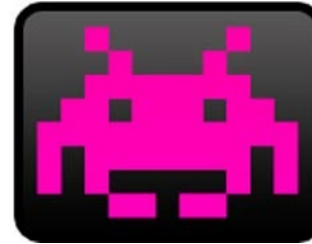
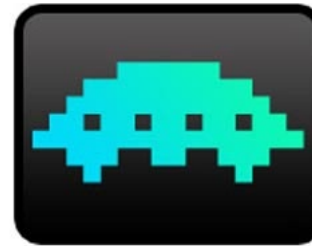


13.0.9.3.2 Mayan Long Count

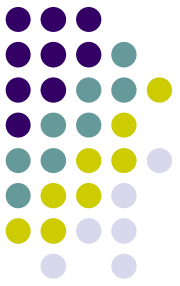


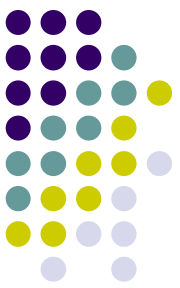
Overview

- C#
 - Have some
 - Gotchas
 - Problems
 - Basics



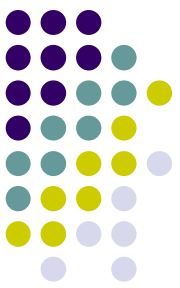
If you saw this before...





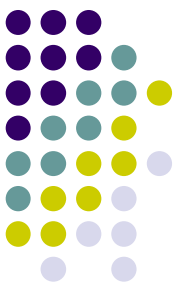
Reference Material

- C# for C++ programmers
 - [https://docs.microsoft.com/en-us/previous-versions/visualstudio/visual-studio-2008/yyaad03b\(v=vs.90\)](https://docs.microsoft.com/en-us/previous-versions/visualstudio/visual-studio-2008/yyaad03b(v=vs.90))
- Articles
 - What You Need to Know to Move from C++ to C#
 - <http://www.cppblog.com/mzty/archive/2005/12/06/1556.html>
 - Ten Traps in C# for C++ Programmers
 - <https://www.thefreelibrary.com/Top+ten+traps+in+C%23+for+C%2B%2B+programmers%3A+Jesse+liberty...+liberty...-a087209372>



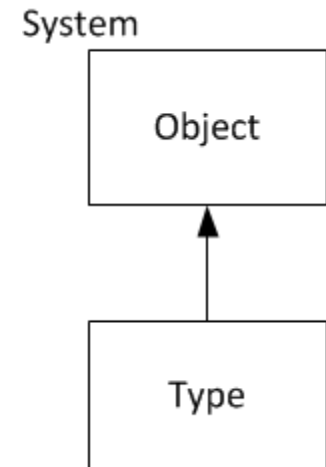
Reference Material

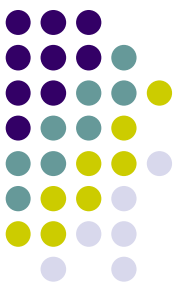
- Microsoft Docs
 - Pretty good
 - <http://msdn.microsoft.com/en-us/library/ms228394%28v=VS.90%29.aspx>
 - Big Daddy
 - [https://msdn.microsoft.com/en-us/library/67ef8sbd\(v=vs.140\).aspx](https://msdn.microsoft.com/en-us/library/67ef8sbd(v=vs.140).aspx)
 - C# Guide from Microsoft
 - <https://docs.microsoft.com/en-us/dotnet/articles/csharp/>



Common Base type

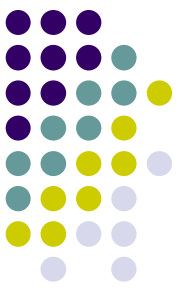
- All types can be treated as if they derive from **object** (***System.Object***)
 - In order to treat an instance of a value type (int, float) as object-derived
 - Instance must be converted to a reference type using a process called **boxing**
- Generally you can forget about this
 - Let the run-time worry about when the conversion
 - Implicit conversion can have **side-effects** that may cause problems





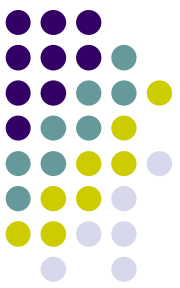
Random notes

- C++ **class** / **structs** are the same except for scope
 - C# structs are very different than C++
- Strings and Chars are Unicode
 - 16-bit **not** 8-bit in C++
- Types of data
 - Value
 - Intrinsic types (e.g. **int**, **char**) are value types.
 - **struct** are also value types.
 - Reference
 - Reference types include classes, arrays and strings
- Basic idea
 - **value** type represents the actual data
 - **reference** type represents a pointer or reference to the data



Boxing

- Boxing
 - Stores value types in the garbage-collected heap.
 - **Implicit** conversion of a value type to the type object or to any interface type implemented by this value type.
 - **Allocates** an object instance on the heap and **copies** the value into the new object.
- Performance
 - boxing and unboxing are computationally expensive processes.
 - When a value type is boxed, a **new** object must be allocated and constructed.
 - To a lesser degree, the **cast** required for unboxing is also expensive computationally.



Boxing example

```
int i = 123;
```

```
// Boxing copies the value of i into object o.
```

```
object o = i;
```

On the stack

i



int i=123;

On the heap

o

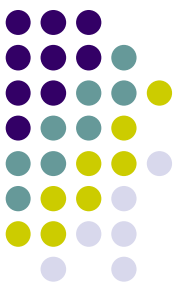


object o=i;

(i boxed)

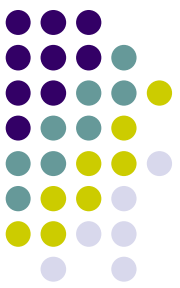


- Implicitly applies the boxing operation on the variable i
- Result
 - Creates an object reference o, on the stack
 - value is a copy



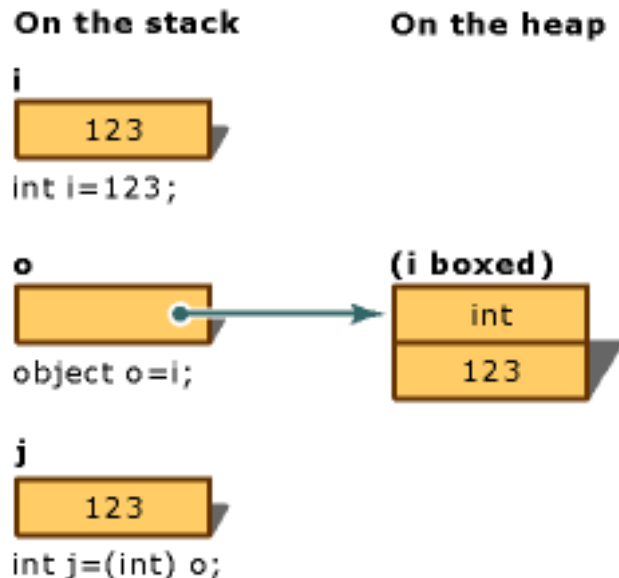
Unboxing

- Unboxing
 - Explicit conversion from the type object to a value type
 - Or an interface type to a value type that implements the interface.
- Unboxing operation:
 - Checking the object instance to make sure that it is a boxed value of the given value type.
 - Copying the value from the instance into the value-type variable.

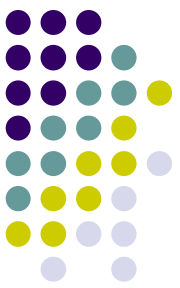


UnBoxing example

```
int i = 123;      // a value type
object o = i;     // boxing
int j = (int)o;   // unboxing
```

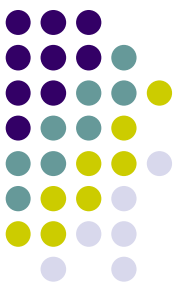


- Successful Unboxing requires:
 - Unboxed item must be a reference to an object that was previously created by boxing an instance of that value type
 - Attempting to unbox null causes a ***NullReferenceException***.
 - Attempting to unbox a reference to an incompatible value type causes an ***InvalidCastException***.



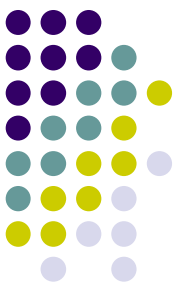
Structs in C#

- Structs share same syntax as classes:
 - struct declaration
 - Generally - fields cannot be initialized
 - Special case - if declared as const or static
- Struct may not declare
 - default constructor
 - destructor



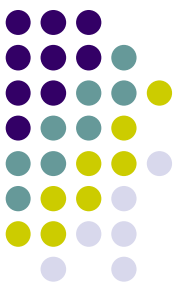
Structs in C#

- Structs are copied on assignment.
 - struct is assigned to a new variable
 - data is copied
 - modification to the new copy does not change original



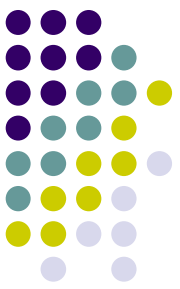
Structs in C#

- Structs are value types and classes are reference types.
 - Unlike classes,
 - structs can be instantiated without using a **new operator**.
 - Structs can declare constructors that have **parameters**.



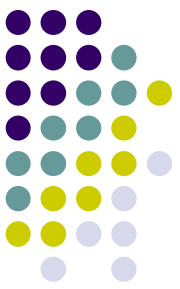
Structs in C#

- Struct cannot inherit from another struct or class, and it cannot be the base of a class.
 - All structs inherit directly from `System.ValueType`, which inherits from `System.Object`.
- A struct can implement interfaces.
- A struct can be used as a nullable type and can be assigned a null value.



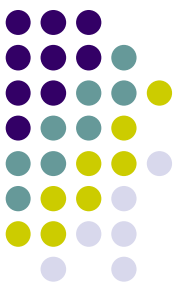
Useful stuff

- Print to the Output Screen
 - `Console.Write()`
- Develop incrementally
 - Big steps will mess with you, hard to find bug
 - Break down into small pieces



Interactions in Class

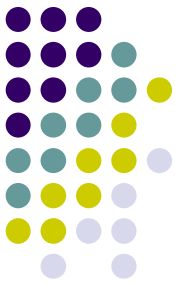
- C# issues
 - Share your experience and knowledge
- Post on the WIKI
 - We will create a sticky list
- Create threads on Piazza



Demo

- C# demos
 - Create project
 - Print
 - Debug
- Demo_1
 - Struct
 - On stack / Dynamic
 - Class
 - Boxing
 - UnBoxing
- Demo_2
 - Linked List
 - Debugging

Questions



- ?????

